# PriceSense Project Report

FINAL-YEAR PROJECT DOCUMENTATION

# PriceSense

## AI Product Price History & Smart Purchase Assistant

TECHNOLOGY STACK

Python 3.11+ · Streamlit · SQLite · SQLAlchemy · Plotly · Ollama · Qwen3 4B · Pytest

IMPLEMENTATION STATUS

**Phases 1–10 complete · offline demo ready**

Implementation report based on the current PriceSense codebase.
Sample prices are deterministic demonstration data, not live marketplace prices.

# Contents

This report is organized as follows. The contents links are clickable in PDF viewers that support document links.

1. Project Overview
2. System Architecture
3. Project Folder Structure
4. Database Implementation
5. Product and Data Collection
6. Sample Data
7. Price Analytics
8. Streamlit Dashboard
9. AI Implementation
10. Target Tracking and Alerts
11. Testing
12. Error Handling and Security
13. Requirements and Setup
14. Complete Demonstration Workflow
15. Current Limitations
16. Testing and Verification Results
17. Future Improvements
18. Conclusion
19. Viva Preparation

# 1. Project Overview

## What PriceSense does

PriceSense is a local web application for inspecting product price history and supporting a purchase decision with transparent historical calculations. A user submits a product URL, reviews stored observations and deterministic analytics, optionally requests a plain-language AI explanation, and can track a target price. An alert is created only when the user explicitly checks the latest stored price and it crosses the target condition.

The working end-to-end source is the offline `SampleAdapter`, which recognizes `sample://demo-001`. Amazon and Flipkart URL recognizers exist, but they do not fetch product pages or prices. Therefore the application does not present unsupported marketplace data as live.

## Problem and objectives

Product prices change over time. A single current price does not show whether a product is relatively high or low compared with its recorded history. PriceSense addresses this by:

- preserving product and price-observation records in SQLite;
- calculating historical statistics deterministically in Python;
- presenting the history and summary in an interactive Streamlit dashboard;
- optionally asking a local language model to explain the computed evidence;
- allowing the user to set a target and explicitly check stored observations for a crossing-based alert.

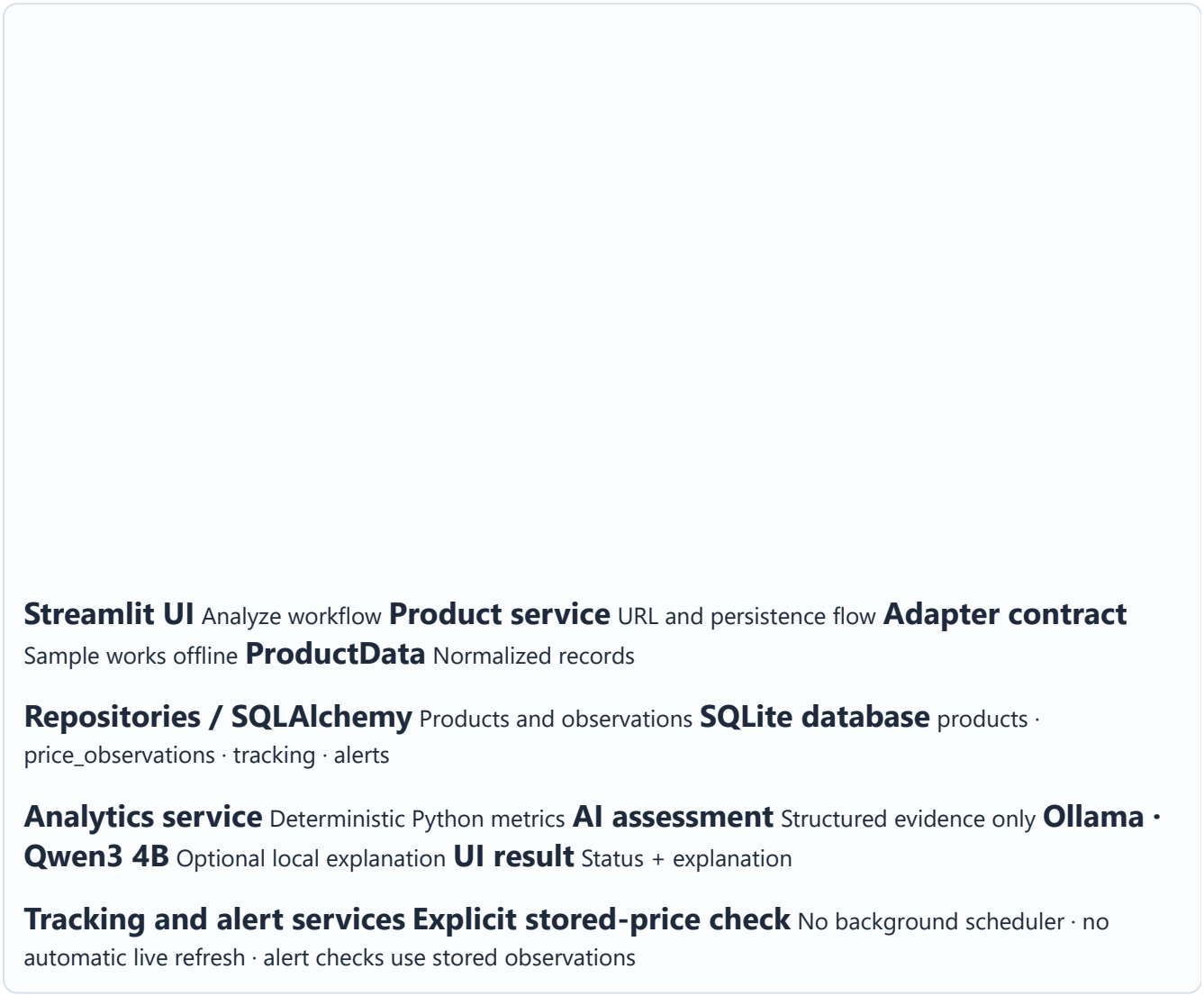
## Why the implementation is useful

The design separates measurement from explanation. Python owns the calculations and status; AI can explain the supplied evidence but cannot invent prices or replace the deterministic status. The offline sample path makes the full product-analysis flow demonstrable without marketplace credentials, network access, or a real product feed.

> **Interpretation boundary:** Analytics describe recorded observations. They are not a prediction that a price will fall and are not a guarantee of future savings.

# 2. System Architecture

PriceSense uses a layered architecture. Streamlit handles user interaction; service modules coordinate product, analytics, AI, tracking, and alert behavior; adapters normalize source data; repositories persist it; and SQLite stores the records. The AI explanation layer receives structured evidence after Python has calculated the metrics.




### Main data flow

1. The user enters a product URL in the Analyze UI.
2. `ProductService` finds the first adapter that recognizes the URL. The sample URL is normalized by `SampleAdapter`; recognized Amazon/Flipkart URLs end with an explicit not-implemented error because there is no permitted configured provider.
3. A new supported product and its returned observations are persisted using repositories. Resolving the same marketplace/source URL again returns the existing product rather than duplicating its initial history.
4. The analytics service reads stored observations and runs the deterministic Python calculations.
5. The AI assessment receives product metadata and a compact structured representation of those calculations. Ollama is optional; deterministic fallback remains available.
6. Tracking and alert services evaluate stored data only when explicitly invoked by the user.

## Responsibility boundaries

| Layer | Responsibility |
| --- | --- |
| Streamlit UI | Input, presentation, clear error messages, and explicit user actions |
| Services | Product resolution, persistence coordination, analytics orchestration, AI assessment, tracking and alert rules |
| Collectors/adapters | Source recognition and normalization into the common product data structure |
| Database/repositories | SQLite initialization, SQLAlchemy models, sessions, and persistence operations |
| Analytics | Deterministic metrics and rule-based status |
| AI | Evidence-grounded explanation; no independent price calculation or prediction |

# 3. Project Folder Structure

| Path | Purpose |
|---|---|
| `app.py` | Streamlit application entry point and page setup |
| `config.py` | Central configuration and environment-backed settings |
| `.env.example` | Safe local configuration example; contains no credentials |
| `.gitignore` | Excludes local environment, virtual environment, caches, and generated local state |
| `requirements.txt` | Runtime and project-test dependencies |
| `README.md` | Setup, operation, architecture, and limitation guidance |
| `collectors/` | Common adapter contract and Sample/Amazon/Flipkart adapter implementations |
| `data/sample_prices.csv` | Deterministic sample observation history for `demo-001` |
| `database/` | SQLAlchemy models, database initialization/session handling, and repositories |
| `services/` | Product, analytics, AI assessment, tracking, and alert orchestration |
| `analytics/` | Price-statistics calculation and analytics service |
| `ai/` | Evidence and response schemas, prompts, assessment validation, and provider interface |
| `ai/providers/` | Local Ollama provider implementation |
| `ui/` | Streamlit Analyze screen and presentation logic |
| `tests/` | Unit, integration, adapter, database, analytics, AI, UI, and tracking/alert tests |

The layers keep UI code separate from business rules. In particular, analytics remain independently testable and are not delegated to the language model.

# 4. Database Implementation

PriceSense uses SQLite for local persistence and SQLAlchemy models/repositories for database access. Database initialization creates the tables from model metadata. The application does not include a schema-migration framework; initialization creates missing tables rather than applying versioned migrations.

## Tables and fields

| Table | Fields |
|---|---|
| `products` | `id` (primary key), `marketplace`, `product_url`, `canonical_product_id`, `title`, `brand`, `model`, `variant`, `currency`, `created_at`, `updated_at` |
| `price_observations` | `id` (primary key), `product_id` (foreign key), `observed_at`, `price`, `currency`, `source`, `availability` |
| `tracking` | `id` (primary key), `product_id` (foreign key), `target_price`, `active`, `created_at` |
| `alerts` | `id` (primary key), `tracking_id` (foreign key), `triggered_at`, `observed_price`, `status` |

## Relationships and integrity

- One product has many price observations.
- One product can have multiple tracking records.
- One tracking record can have multiple alert records over time.
- `price_observations.product_id` and `tracking.product_id` reference `products.id`.
- `alerts.tracking_id` references `tracking.id`.
- The foreign keys use `ON DELETE CASCADE`; ORM relationships also apply delete-orphan cascades.
- The product marketplace and URL pair has a named uniqueness constraint to avoid duplicate product identities.
- Foreign-key columns are indexed for lookup.

**products** id · PK marketplace · product_url canonical_product_id title · brand · model · variant currency · created_at · updated_at

**price_observations** id · PK product_id · FK → products.id observed_at · price · currency source · availability

**tracking** id · PK product_id · FK → products.id target_price · active · created_at

**alerts** id · PK tracking_id · FK triggered_at · observed_price · status 1 to many 1 to many 1 to many

The local database retains products, initial sample observations, user-created tracking records, and alert history across application runs. Tests use isolated database setups so they do not depend on a user's local data.

# 5. Product and Data Collection

## Adapter contract

The service layer depends on a common `ProductDataAdapter` interface rather than marketplace-specific code. An adapter reports whether it supports a URL and, when implemented, returns normalized product metadata and price observations. This boundary allows a permitted data source to be added without moving marketplace/API logic into the UI.

## Implemented adapters

| Adapter | Current behavior |
|---|---|
| `SampleAdapter` | Fully implemented offline adapter for known `sample://` identifiers; loads the bundled CSV, normalizes the data, and returns product metadata plus chronological observations |
| `AmazonAdapter` | Recognizes matching Amazon host text, but `fetch_product()` raises `LiveMarketplaceNotImplementedError`; it performs no network request |
| `FlipkartAdapter` | Recognizes matching Flipkart host text, but `fetch_product()` raises `LiveMarketplaceNotImplementedError`; it performs no network request |

Adapter resolution checks Sample, Amazon, then Flipkart. An unsupported URL raises an unsupported-URL error. A recognized marketplace URL does not fall through to sample data: doing so would falsely label synthetic prices as live marketplace observations.

## Source limitations

No permitted live source or credentials were available/configured. No marketplace HTML scraping, browser automation, stealth behavior, or access-control circumvention is implemented. The live marketplace classes are recognition placeholders only; they are not product APIs and do not fetch a current price.

# 6. Sample Data

The only bundled sample catalog identity is `demo-001`, available as `sample://demo-001`.

| Property | Value |
|---|---|
| Title | PriceSense Demo Wireless Headphones (SAMPLE DATA - not a live price) |
| Brand | PriceSense Demo |
| Model | Demo-001 |
| Variant | Black / Over-Ear |
| Marketplace/source | `sample` |
| Currency | INR |
| Availability | `in_stock` |
| Source URL | `sample://demo-001` |

The adapter loads these eight deterministic rows from `data/sample_prices.csv`, orders them chronologically, and uses the last row as the current sample price:

| Observation date (UTC) | Price |
|---|---:|
| 2026-09-01 | ₹25,999 |
| 2026-09-03 | ₹25,499 |
| 2026-09-05 | ₹24,999 |
| 2026-09-10 | ₹23,999 |
| 2026-09-15 | ₹22,999 |
| 2026-09-20 | ₹23,999 |
| 2026-09-25 | ₹22,499 |
| 2026-09-30 | ₹22,499 |

These are demonstration observations, not prices fetched from a retailer. Re-running the sample analysis is deterministic. The product title, UI labeling, and observation source distinguish the sample from live data.

# 7. Price Analytics

All price calculations are performed by Python in the analytics layer. The AI does not calculate these values. Invalid observations are excluded when their price is missing, non-finite, negative, or their timestamp is missing. Valid observations are ordered by timestamp; the newest valid one supplies the current price.

Let the valid prices be (p_1, p_2, , p_n), ordered oldest to newest.

| Metric | Implemented calculation |
|---|---|
| Current price | Price of the newest valid observation |
| Lowest price | ((p_i)) |
| Highest price | ((p_i)) |
| Average | () |
| Median | Standard median of the sorted valid prices |
| Observation count | Number of valid observations |
| First / last observation | Earliest / latest valid observation timestamp |
| Recent average | Arithmetic mean of the most recent configured window; the window shrinks when fewer observations exist |
| Price position | (); undefined (`None`) when the historical range is zero |
| Difference from comparison | (current-comparison) |
| Percentage difference | (); undefined when the comparison value is zero or unavailable |

## Trend

Trend uses the configured recent trend window. With at least two points and a non-zero starting price:

[ = ]

When the starting price is zero, the implementation uses the raw difference sign rather than dividing by zero. Fewer than two points results in `INSUFFICIENT_DATA`.

## Rule-based status

Status compares the current price with the historical average. At least three valid observations are required by the current default policy. A zero historical range is also treated as insufficient data.

| Condition | Status |
|---|---|
| Fewer than the minimum observations, or no historical price range | `INSUFFICIENT_DATA` |

| Condition | Status |
| --- | --- |
| Current price is at least 10% below average (percentage difference ≤ −10%) | `LOW` |
| Current price is at least 10% above average (percentage difference ≥ +10%) | `HIGH` |
| Otherwise | `AVERAGE` |

The status thresholds and minimum count are configuration policy; they are deterministic rules, not a model opinion.

## Edge cases and sample result

- Empty or wholly invalid observations do not produce fabricated metrics.
- One observation can supply a current price but is insufficient for a trend/status requiring more data.
- Identical prices have a zero-width range, so price position is undefined and status is insufficient.
- A zero comparison denominator yields no percentage difference instead of division by zero.
- Negative and non-finite prices are excluded.

For the bundled sample history, the current price is ₹22,499, low ₹22,499, high ₹25,999, average ₹24,061.50, median ₹23,999, recent average ₹22,999, and observation count 8. The recent trend is approximately −6.25%; price position is 0% because the current price equals the observed low. The current price is about 6.5% below the average, which is within the configured ±10% band, so the deterministic status is `AVERAGE`.

# 8. Streamlit Dashboard

The Analyze screen provides a single workflow:

1. Enter a product URL and select **Analyze**.
2. Review normalized product details and the visible SAMPLE / DEMO DATA label for the bundled adapter.
3. Review metrics for current price, low, high, average, median, recent average, trend, price position, and observation count.
4. Inspect the Plotly price-history chart.
5. Read the deterministic rule-based status and its evidence.
6. Optionally request an AI explanation, or use the deterministic fallback if local Ollama is unavailable.
7. Set or review a target-price tracking record.
8. Select **Check stored price** to explicitly evaluate the most recent persisted observation.
9. Review the target status and alert history.

The dashboard does not schedule checks. Rerendering the page or changing the target does not itself constitute a stored-price alert check. Invalid or unavailable sources are reported to the user without showing sample prices as a substitute for a requested marketplace product.

# 9. AI Implementation

## Provider and evidence flow

The implemented provider is local Ollama, using the configured `qwen3:4b` model through Ollama's `/api/chat` interface. The request is non-streaming, asks for JSON, disables model thinking, uses temperature zero, and applies a configured timeout. AI is optional; the rest of the application remains useful without it.

Before the provider is called, Python builds structured evidence from the product and computed analytics. Evidence includes product metadata; current, low, high, average, and median prices; observation count and date range; recent average; trend; price position; comparison values; deterministic status; and its reason. The model does not receive responsibility for retrieving marketplace prices or computing authoritative metrics.

## Response validation and deterministic authority

The expected response is a JSON object with exactly four fields:

```
{
  "status": "LOW | AVERAGE | HIGH | INSUFFICIENT_DATA",
  "summary": "A concise explanation",
  "evidence": ["A statement grounded in supplied metrics"],
  "caveat": "A limitation or uncertainty"
}
```

The response schema checks the allowed status, required non-empty text, and evidence shape. Content referring to transactions is rejected because this application does not execute or verify purchases. A provider or malformed/invalid response leads to deterministic fallback. If a structurally valid response disagrees with Python's status, the service makes one corrective retry; failure to correct the mismatch returns the deterministic fallback.

The fallback is generated from Python evidence, is labeled `Deterministic fallback`, and does not predict future prices. The UI reuses a prior assessment when the evidence fingerprint has not changed. This avoids unnecessary repeated calls for identical evidence.

The provider response is limited to validated content; model thinking/reasoning is disabled at the Ollama request level and is not presented as an application answer. Deterministic Python status remains the source of truth even when the AI explanation is available.

# 10. Target Tracking and Alerts

## Tracking status

A target must be a finite numeric value greater than zero. Boolean values, NaN, infinity, zero, and negative values are rejected. Tracking records are associated with products and can be retrieved or deactivated; deactivation preserves alert history.

For an active tracking record, status is computed from the latest stored observation (ordered by observation time, with ID as a tie-breaker):

- **Target reached:** `latest_price <= target_price`.
- **Target difference:** `latest_price - target_price`.
- Missing observations mean there is no current price to evaluate.

No live price is fetched as part of tracking. The user explicitly invokes **Check stored price** to evaluate the persisted observation.

## Crossing-based alert state

| Event | Result |
|---|---|
| First explicit check at or below target | Create a `triggered` alert with the observed price |
| Another check while price remains at or below target | Do not create a duplicate |
| A later check finds price above target | Change the current triggered alert state to `reset` |
| A subsequent check again finds price at or below target | Create a new `triggered` alert |
| Tracking is inactive | Skip evaluation and create no alert |

This is an explicit state transition, not a scheduled notification system. Alert history exposes prior trigger/reset records; no email, SMS, push delivery, or background worker is implemented.

# 11. Testing

The project uses Pytest. Tests cover individual rules and cross-layer behavior; database-backed tests isolate persistence instead of depending on a user's database or a marketplace account.

| Test area | Coverage represented in the suite |
|---|---|
| Database | Initialization, tables, product and observation persistence, tracking/alert persistence |
| Adapters and product service | Sample URL support, deterministic normalization, product resolution, unsupported and not-implemented marketplace behavior |
| Analytics | Empty/single/identical observations, invalid values, thresholds, averages, medians, trend, position, and insufficient-data paths |
| AI | Structured response validation, status consistency/corrective retry, provider errors, malformed responses, and deterministic fallback |
| Tracking and alerts | CRUD, target validation, reached/not reached, inactive records, persistence, duplicate prevention, reset, and recross |
| UI and integration | Sample and unsupported URL behavior plus an end-to-end flow through storage, analytics, AI, tracking, and alerts |

The final verification result for this project is recorded in Section 16.

# 12. Error Handling and Security

## User-facing failure behavior

- Blank or malformed input is rejected with a clear request to enter a valid URL.
- Unsupported URLs direct the user toward the supported sample URL.
- A recognized Amazon/Flipkart URL explains that no live price was fetched because no live adapter is configured.
- Missing sample data is reported as unavailable; it is not replaced with a made-up price.
- Missing products, missing observations, invalid targets, inactive tracking, and persistence failures are handled in the appropriate service/UI flow.
- Ollama connection, timeout, model, HTTP, or malformed-output failures use the deterministic explanation fallback where possible.
- Internal exception details are logged rather than exposed as raw UI output.

## Security and privacy boundaries

- `.env` is excluded from version control; `.env.example` is for safe local configuration and placeholders, not real credentials.
- No marketplace API key, password, cookie, or token is embedded in source.
- Ollama is a local provider; AI is optional and no marketplace credentials are passed to it.
- The SQLite file is local and is not encrypted by the application.
- No login or multi-user access-control system is implemented.
- PriceSense does **not** bypass CAPTCHA, authentication, bot protection, rate limits, paywalls, robots restrictions, or other marketplace access controls.
- The code does not scrape Amazon or Flipkart pages and does not use browser automation or stored browser session data.

# 13. Requirements and Setup

## Requirements

- Python 3.11 or newer.
- A Windows PowerShell terminal for the commands below (the Python application itself uses cross-platform libraries).
- Dependencies installed from the repository's `requirements.txt`.
- Optional: Ollama installed locally and the `qwen3:4b` model available for AI explanations.

SQLite is provided through Python's SQLite support. Live marketplace credentials are not required because no live provider is implemented.

## Virtual environment and dependencies

Run these commands from the project directory:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
Copy-Item .env.example .env
```

The local `.env` is ignored by Git. Keep any private values local; do not copy secrets into `.env.example`.

## Optional Ollama setup

Install Ollama using its official distribution, then download the configured model:

```
ollama pull qwen3:4b
```

Ensure the local Ollama service is running before requesting an AI explanation. If Ollama is unavailable, the deterministic fallback is used; the sample analytics and tracking workflow still work.

## Run tests and start the application

```
.\.venv\Scripts\python.exe -m pytest -v
.\.venv\Scripts\python.exe -m streamlit run app.py
```

The application can also be started with `streamlit run app.py` after activating the virtual environment.

# 14. Complete Demonstration Workflow

The following workflow uses the bundled sample and does not require a live marketplace:

1. Start Streamlit and enter `sample://demo-001` in the Analyze URL field.
2. Select **Analyze**. Confirm that the product title and SAMPLE / DEMO DATA indicators are visible.
3. Review the eight stored sample observations and the chart. The current displayed sample price is ₹22,499 INR.
4. Review deterministic values: low ₹22,499; high ₹25,999; average ₹24,061.50; median ₹23,999; recent average ₹22,999; approximate recent trend −6.25%; position 0%; status `AVERAGE`.
5. Request the AI explanation. When Ollama/Qwen3 4B is running, the explanation should discuss the evidence. When it is not, confirm the labeled deterministic fallback. Neither result changes the Python-computed status.
6. Add a tracking record with a valid target. A target of ₹22,499 demonstrates the reached condition because the latest stored price is less than or equal to the target.
7. Select **Check stored price**. The explicit check creates a triggered alert for the stored observation.
8. Select **Check stored price** again without changing the observations. Confirm that no duplicate alert is created.
9. Review alert history and tracking status.
10. For an unsupported marketplace URL, confirm the user-facing unavailable-source message and that the sample product is not substituted.

The bundled observations demonstrate trigger and duplicate prevention. The reset/recross transition is covered by tests with controlled observations; the bundled sample series does not contain an above-target observation after its first target crossing for the example target, so it should not be represented as a sample-data event.

# 15. Current Limitations

- No permitted/configured live marketplace source is available; Amazon and Flipkart adapters only recognize URLs and report that fetching is not implemented.
- The sample product and prices are static deterministic demonstration data.
- There is no automatic scheduled price collection, live refresh, or background alert evaluation.
- Alerts are records in local SQLite only; there is no email, SMS, push, or external notification delivery.
- The local SQLite database is not encrypted by the application.
- There is no authentication or multi-user access control.
- Analytics describe recorded history; they do not forecast prices or guarantee future reductions.
- AI explanations depend on local Ollama availability, but deterministic fallback is available.
- Database schema creation is handled by SQLAlchemy metadata; a versioned migration system is not included.

# 16. Testing and Verification Results

The final project verification ran the complete suite from the project virtual environment:

```
Command: .\.venv\Scripts\python.exe -m pytest -v
Result:  117 passed in 6.79s
Failed:  0
```

The suite collected 117 tests and completed successfully. This verifies the existing Phase 1–10 implementation at the time of documentation. The PDF-generation process itself does not modify or test application source code.

Previously completed final-polish verification also manually exercised the Streamlit sample flow, invalid input, unsupported Amazon URL behavior, AI success/fallback, target tracking, explicit stored-price checks, duplicate prevention, and alert history. The local Ollama/Qwen3 path and unavailable-provider fallback were both checked. No live marketplace behavior was tested or claimed because no permitted live source is configured.

# 17. Future Improvements

The following are possible future work only; they are not implemented features:

- Integrate one marketplace or affiliate API only after its access terms and authorization are verified.
- Add an opt-in scheduler for permitted sources and make collection times visible to users.
- Add user-configurable notification delivery, with explicit consent and secure credential handling.
- Add database backup and encryption options appropriate to the deployment environment.
- Add authentication and authorization if the application becomes multi-user.
- Expand the demo catalog or provide a separately labeled synthetic dataset that demonstrates alert reset and recross transitions.
- Add additional accessibility checks and automated browser-level UI coverage as the interface evolves.

# 18. Conclusion

PriceSense is a working, layered demonstration of product-history analysis, deterministic price statistics, locally hosted AI explanation, and explicit target-price tracking. Its offline sample path demonstrates the application without implying access to real marketplace prices. Python remains authoritative for metrics and status; AI contributes only an evidence-grounded explanation. The completed test suite and user-facing fallbacks support a reliable project demonstration while the absence of live data collection, scheduling, and external notifications remains clearly documented.

# 19. Viva Preparation

| Likely question | Concise answer |
| --- | --- |
| Why SQLite? | It provides persistent relational storage without a separate database server, which suits a local demonstration and makes test databases easy to isolate. |
| Why use adapters? | Adapters isolate source-specific recognition and normalization behind a shared contract. Services and UI do not need marketplace-specific parsing logic. |
| Why deterministic analytics? | Price metrics must be reproducible and auditable. The same stored observations produce the same calculations and rule-based status. |
| Why use AI after analytics? | AI can explain structured results in accessible language, while Python retains control of calculations and authoritative status. |
| Why Ollama and Qwen3 4B? | The implementation uses a local Ollama provider and a compact Qwen3 4B model for optional explanations without requiring a hosted AI API. |
| How is price position calculated? | `(current - lowest) / (highest - lowest) * 100`. It is undefined for a zero-range history. |
| How do alerts avoid duplicates? | An active trigger remains the current crossing state while price is at or below target. Repeated checks do not add another alert until a check above target resets it. |
| What happens when AI fails? | The assessment service returns a labeled deterministic explanation from Python-computed evidence. |
| What happens for unsupported marketplaces? | The UI reports that no live source is implemented/configured. It does not show the sample price as if it were live. |
| What are the main limitations? | No permitted live provider, no scheduler, no external notifications, a static demo dataset, local unencrypted SQLite, and no prediction or guarantee of future price movement. |
| What could be improved later? | A properly authorized provider, opt-in collection and notification services, multi-user security, database protection, and a separate reset/recross demonstration dataset. |